Chapter ML:IV

IV. Neural Networks

- Perceptron Learning
- Gradient Descent
- Multilayer Perceptron
- Radial Basis Functions

©STEIN 2005-2020

Gradient Descent

Classification Error

Gradient descent considers the hyperplane distance and will converge even if X_1 and X_0 cannot be separated by a hyperplane. However, this convergence process is of an asymptotic nature and no finite iteration bound can be stated.

Gradient descent applies the so-called delta rule, which will be derived in the following. The delta rule forms the basis of the backpropagation algorithm.

©STEIN 2005-2020

Gradient Descent

Classification Error

Gradient descent considers the hyperplane distance and will converge even if X_1 and X_0 cannot be separated by a hyperplane. However, this convergence process is of an asymptotic nature and no finite iteration bound can be stated.

Gradient descent applies the so-called delta rule, which will be derived in the following. The delta rule forms the basis of the backpropagation algorithm.

Consider the linear perceptron *without* a threshold function:

$$y(\mathbf{x}) \;=\; \mathbf{w}^T\mathbf{x} \;=\; \sum_{j=0}^{p} w_j x_j \qquad \text{[Heaviside]}$$

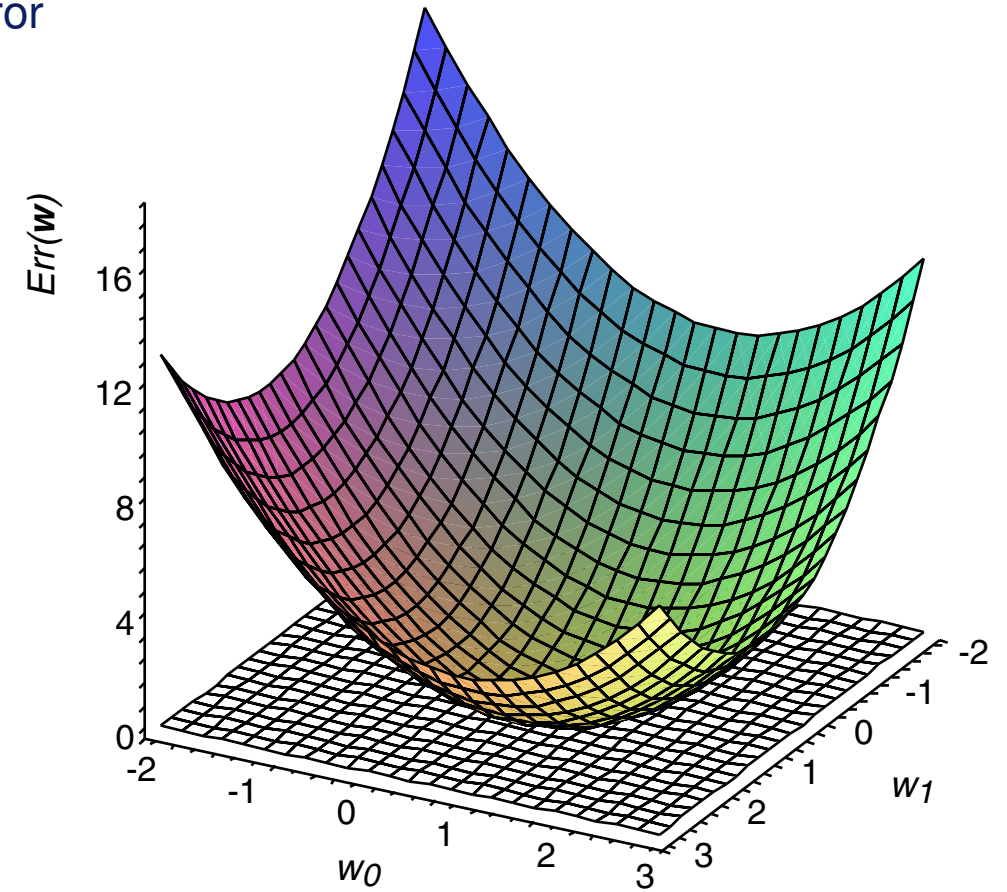
The classification error $Err(\mathbf{w})$ of a weight vector (= hypothesis) $\mathbf{w}$ with regard to D can be defined as follows:

$$Err(\mathbf{w}) \;=\; \frac{1}{2}\,\mathrm{RSS}(\mathbf{w}) \;=\; \frac{1}{2}\sum_{(\mathbf{x},c(\mathbf{x}))\in D} \big(c(\mathbf{x}) - y(\mathbf{x})\big)^2 \qquad \text{[Singleton error]}$$

©STEIN 2005-2020

Gradient Descent

Classification Error



The gradient of $Err(\mathbf{w})$, $\nabla Err(\mathbf{w})$, defines the steepest ascent or descent:

$$\nabla Err(\mathbf{w}) \;=\; \left(\frac{\partial Err(\mathbf{w})}{\partial w_0}, \frac{\partial Err(\mathbf{w})}{\partial w_1}, \ldots, \frac{\partial Err(\mathbf{w})}{\partial w_p} \right)$$

©STEIN 2005-2020

Gradient Descent

Weight Adaptation

$$\mathbf{w} \leftarrow \mathbf{w} + \Delta\mathbf{w} \quad \text{where} \quad \Delta\mathbf{w} = -\eta \nabla \mathit{Err}(\mathbf{w})$$ [*PT* Algorithm]

Componentwise ($j = 0, \ldots, p$) weight adaptation:

$$w_j \leftarrow w_j + \Delta w_j \quad \text{where} \quad \Delta w_j = -\eta \frac{\partial}{\partial w_j} \mathit{Err}(\mathbf{w})$$

©STEIN 2005-2020

Gradient Descent

Weight Adaptation

$$\mathbf{w} \leftarrow \mathbf{w} + \Delta\mathbf{w} \quad \text{where} \quad \Delta\mathbf{w} = -\eta \nabla \mathit{Err}(\mathbf{w})$$ [*PT* Algorithm]

Componentwise ($j = 0, \ldots, p$) weight adaptation:

$$w_j \leftarrow w_j + \Delta w_j \quad \text{where} \quad \Delta w_j = -\eta \frac{\partial}{\partial w_j} \mathit{Err}(\mathbf{w})$$

$$\frac{\partial}{\partial w_j} \mathit{Err}(\mathbf{w}) = \frac{\partial}{\partial w_j} \frac{1}{2} \sum_{(\mathbf{x}, c(\mathbf{x})) \in D} (c(\mathbf{x}) - y(\mathbf{x}))^2 = \frac{1}{2} \sum_{(\mathbf{x}, c(\mathbf{x})) \in D} \frac{\partial}{\partial w_j} (c(\mathbf{x}) - y(\mathbf{x}))^2$$

©STEIN 2005-2020

Gradient Descent

Weight Adaptation

$$\mathbf{w} \leftarrow \mathbf{w} + \Delta\mathbf{w} \quad \text{where} \quad \Delta\mathbf{w} = -\eta \nabla \mathit{Err}(\mathbf{w})$$ [*PT* Algorithm]

Componentwise ($j = 0, \ldots, p$) weight adaptation:

$$w_j \leftarrow w_j + \Delta w_j \quad \text{where} \quad \Delta w_j = -\eta \frac{\partial}{\partial w_j} \mathit{Err}(\mathbf{w})$$

$$\frac{\partial}{\partial w_j} \mathit{Err}(\mathbf{w}) = \frac{\partial}{\partial w_j} \frac{1}{2} \sum_{(\mathbf{x}, c(\mathbf{x})) \in D} (c(\mathbf{x}) - y(\mathbf{x}))^2 = \frac{1}{2} \sum_{(\mathbf{x}, c(\mathbf{x})) \in D} \frac{\partial}{\partial w_j} (c(\mathbf{x}) - y(\mathbf{x}))^2$$

$$= \frac{1}{2} \sum_{(\mathbf{x}, c(\mathbf{x})) \in D} 2(c(\mathbf{x}) - y(\mathbf{x})) \cdot \frac{\partial}{\partial w_j} (c(\mathbf{x}) - y(\mathbf{x}))$$

©STEIN 2005-2020

Gradient Descent

Weight Adaptation

$$\mathbf{w} \leftarrow \mathbf{w} + \Delta\mathbf{w} \quad \text{where} \quad \Delta\mathbf{w} = -\eta \nabla \mathit{Err}(\mathbf{w})$$ [*PT* Algorithm]

Componentwise ($j = 0, \ldots, p$) weight adaptation:

$$w_j \leftarrow w_j + \Delta w_j \quad \text{where} \quad \Delta w_j = -\eta \frac{\partial}{\partial w_j} \mathit{Err}(\mathbf{w})$$

$$\frac{\partial}{\partial w_j} \mathit{Err}(\mathbf{w}) = \frac{\partial}{\partial w_j} \frac{1}{2} \sum_{(\mathbf{x}, c(\mathbf{x})) \in D} (c(\mathbf{x}) - y(\mathbf{x}))^2 = \frac{1}{2} \sum_{(\mathbf{x}, c(\mathbf{x})) \in D} \frac{\partial}{\partial w_j} (c(\mathbf{x}) - y(\mathbf{x}))^2$$

$$= \frac{1}{2} \sum_{(\mathbf{x}, c(\mathbf{x})) \in D} 2(c(\mathbf{x}) - y(\mathbf{x})) \cdot \frac{\partial}{\partial w_j} (c(\mathbf{x}) - y(\mathbf{x}))$$

$$= \sum_{(\mathbf{x}, c(\mathbf{x})) \in D} (c(\mathbf{x}) - \mathbf{w}^T \mathbf{x}) \cdot \frac{\partial}{\partial w_j} (c(\mathbf{x}) - \mathbf{w}^T \mathbf{x})$$

©STEIN 2005-2020

Gradient Descent

Weight Adaptation

$$\mathbf{w} \leftarrow \mathbf{w} + \Delta\mathbf{w} \quad \text{where} \quad \Delta\mathbf{w} = -\eta \nabla \mathit{Err}(\mathbf{w})$$ [*PT* Algorithm]

Componentwise ($j = 0, \ldots, p$) weight adaptation:

$$w_j \leftarrow w_j + \Delta w_j \quad \text{where} \quad \Delta w_j = -\eta \frac{\partial}{\partial w_j} \mathit{Err}(\mathbf{w})$$

$$\begin{aligned}
\frac{\partial}{\partial w_j} \mathit{Err}(\mathbf{w}) &= \frac{\partial}{\partial w_j} \frac{1}{2} \sum_{(\mathbf{x}, c(\mathbf{x})) \in D} (c(\mathbf{x}) - y(\mathbf{x}))^2 = \frac{1}{2} \sum_{(\mathbf{x}, c(\mathbf{x})) \in D} \frac{\partial}{\partial w_j} (c(\mathbf{x}) - y(\mathbf{x}))^2 \\
&= \frac{1}{2} \sum_{(\mathbf{x}, c(\mathbf{x})) \in D} 2(c(\mathbf{x}) - y(\mathbf{x})) \cdot \frac{\partial}{\partial w_j} (c(\mathbf{x}) - y(\mathbf{x})) \\
&= \sum_{(\mathbf{x}, c(\mathbf{x})) \in D} (c(\mathbf{x}) - \mathbf{w}^T \mathbf{x}) \cdot \frac{\partial}{\partial w_j} (c(\mathbf{x}) - \mathbf{w}^T \mathbf{x}) \\
&= \sum_{(\mathbf{x}, c(\mathbf{x})) \in D} (c(\mathbf{x}) - \mathbf{w}^T \mathbf{x})(-x_j)
\end{aligned}$$

©STEIN 2005-2020

Gradient Descent

Weight Adaptation

$$\mathbf{w} \leftarrow \mathbf{w} + \Delta\mathbf{w} \quad \text{where} \quad \Delta\mathbf{w} = -\eta \nabla \mathit{Err}(\mathbf{w})$$ [*PT* Algorithm]

Componentwise ($j = 0, \ldots, p$) weight adaptation:

$$w_j \leftarrow w_j + \Delta w_j \quad \text{where} \quad \Delta w_j = -\eta \frac{\partial}{\partial w_j} \mathit{Err}(\mathbf{w}) = \eta \sum_{(\mathbf{x}, c(\mathbf{x})) \in D} (c(\mathbf{x}) - \mathbf{w}^T\mathbf{x}) \cdot x_j$$

$$\frac{\partial}{\partial w_j} \mathit{Err}(\mathbf{w}) = \frac{\partial}{\partial w_j} \frac{1}{2} \sum_{(\mathbf{x}, c(\mathbf{x})) \in D} (c(\mathbf{x}) - y(\mathbf{x}))^2 = \frac{1}{2} \sum_{(\mathbf{x}, c(\mathbf{x})) \in D} \frac{\partial}{\partial w_j} (c(\mathbf{x}) - y(\mathbf{x}))^2$$

$$= \frac{1}{2} \sum_{(\mathbf{x}, c(\mathbf{x})) \in D} 2(c(\mathbf{x}) - y(\mathbf{x})) \cdot \frac{\partial}{\partial w_j} (c(\mathbf{x}) - y(\mathbf{x}))$$

$$= \sum_{(\mathbf{x}, c(\mathbf{x})) \in D} (c(\mathbf{x}) - \mathbf{w}^T\mathbf{x}) \cdot \frac{\partial}{\partial w_j} (c(\mathbf{x}) - \mathbf{w}^T\mathbf{x})$$

$$= \sum_{(\mathbf{x}, c(\mathbf{x})) \in D} (c(\mathbf{x}) - \mathbf{w}^T\mathbf{x})(-x_j)$$

©STEIN 2005-2020

Gradient Descent

Weight Adaptation: Batch Gradient Descent [Algorithms: *IGD* *PT*]

Algorithm:	*BGD*	Batch Gradient Descent
Input:	D	Training examples $(\mathbf{x}, c(\mathbf{x}))$ with $\lvert\mathbf{x}\rvert = p+1,\ c(\mathbf{x}) \in \{0,1\}$. // $c(\mathbf{x}) \in \{-1,1\}$
	η	Learning rate, a small positive constant.
Internal:	$y(D)$	Set of $y(\mathbf{x})$-values computed from the elements $\mathbf{x}$ in D given some $\mathbf{w}$.
Output:	$\mathbf{w}$	Weight vector.

BGD(D, η)

```
initialize_random_weights(w),  t = 0
REPEAT
   t = t + 1
   Δw = 0
   FOREACH (x, c(x)) ∈ D DO
      error = c(x) − w^T x
      Δw = Δw + η · error · x
   ENDDO
   w = w + Δw
UNTIL(convergence(D, y(D)) OR t > t_max)
return(w)
```


©STEIN 2005-2020

Remarks:

- $\Delta\mathbf{w} \sim -\nabla Err(\mathbf{w})$; i.e., proportional to "$-$" and not to "$+$" to descend to the minimum.

- Each BGD iteration "`REPEAT ... UNTIL`" corresponds to finding the direction of steepest error descent as $-\nabla Err(\mathbf{w_t}) = \sum_{(\mathbf{x},c(\mathbf{x}))\in D} \left(c(\mathbf{x}) - \mathbf{w}_t^T\mathbf{x}\right) \cdot \mathbf{x}$ and updating $\mathbf{w_t}$ by taking a step of length η in this direction.

- Using a constant step size η can severely impair the speed of convergence.
  When taking the optimal step size $\eta_t := \text{argmin}_\eta\, Err(\mathbf{w}_t - \eta \cdot \nabla Err(\mathbf{w_t}))$ at each iteration t, it can be shown that gradient descent has merely a linear rate of convergence. [Meza 2010]

- The *convergence* function may compute the global error, either quantified as the sum of the squared residuals, $Err(\mathbf{w}_t)$, or as the norm of the error gradient, $||\nabla Err(\mathbf{w_t})||$, and compare it to some small positive bound ε.

©STEIN 2005-2020

Gradient Descent

Weight Adaptation: Delta Rule

The weight adaptation in the *BGD* Algorithm is set-based: before modifying a weight component in $\mathbf{w}$, the total error of *all* examples (the "batch") is computed.

Weight adaptation with regard to a *single* example $(\mathbf{x}, c(\mathbf{x})) \in D$:

$$\Delta\mathbf{w} \;=\; \eta \cdot (c(\mathbf{x}) - \mathbf{w}^T\mathbf{x}) \cdot \mathbf{x}$$

This adaptation rule is known under different names:

- delta rule
- Widrow-Hoff rule
- adaline rule
- least mean squares (LMS) rule

The classification error $Err_d(\mathbf{w})$ of a weight vector (= hypothesis) $\mathbf{w}$ with regard to a *single* example $d \in D$, $d = (\mathbf{x}, c(\mathbf{x}))$, is given as:

$$Err_d(\mathbf{w}) \;=\; \frac{1}{2}\,(c(\mathbf{x}) - \mathbf{w}^T\mathbf{x})^2 \qquad \text{[Batch error]}$$

©STEIN 2005-2020

Gradient Descent

Weight Adaptation: Incremental Gradient Descent [Algorithms: *BGD* *PT* *LMS*]

Algorithm:	*IGD*	Incremental Gradient Descent
Input:	D	Training examples $(\mathbf{x}, c(\mathbf{x}))$ with $\lvert\mathbf{x}\rvert = p+1,\ c(\mathbf{x}) \in \{0,1\}$. // $c(\mathbf{x}) \in \{-1,1\}$
	η	Learning rate, a small positive constant.
Internal:	$y(D)$	Set of $y(\mathbf{x})$-values computed from the elements $\mathbf{x}$ in D given some $\mathbf{w}$.
Output:	$\mathbf{w}$	Weight vector.

IGD(D, η)

```
initialize_random_weights(w),  t = 0
REPEAT
  t = t + 1
  FOREACH (x, c(x)) ∈ D DO
    error = c(x) − w^T x
    Δw = η · error · x
    w = w + Δw
  ENDDO
UNTIL(convergence(D, y(D)) OR t > t_max)
return(w)
```


©STEIN 2005-2020

Remarks:

- The sequence of incremental weight adaptations approximates the gradient descent of the batch approach. If η is chosen sufficiently small, this approximation can happen at arbitrary accuracy.
- The computation of the total error of batch gradient descent enables larger weight adaptation increments.
- Compared to batch gradient descent, the example-based weight adaptation of incremental gradient descent can better avoid getting stuck in a local minimum of the error function.
- A related method is *stochastic* gradient descent, which estimates the gradient from a randomly selected subset of the data.

©STEIN 2005-2020

Remarks (continued):

- When, as is done here, the residual sum of squares, RSS, is chosen as error (loss) function, the incremental gradient descent algorithm [*IGD*] is very similar to the least mean squares algorithm [*LMS*].

- The incremental gradient descent algorithm [*IGD*] looks similar to the perceptron training algorithm [*PT*], since these algorithms differ only in the error computation (Line 5) where the latter applies the Heaviside function. However, this subtle syntactic difference is a significant conceptual difference, entailing a number of consequences:
  - Gradient descent is a regression approach and exploits the residua, which are provided by an error function of choice, and whose differential is evaluated to control the hyperplane movement.
  - The *PT* algorithm is not based on residuals (in the $(p+1)$-dimensional input-output-space) but refers to the input space only, where it simply evaluates the side of the hyperplane as a binary feature (correct side or not).
  - Provided linear separability, the *PT* algorithm will converge within a finite number of iterations, which, however, cannot be guaranteed for gradient descent.
  - Gradient descent may converge even if the data is not linearly separable.
  - Data sets can be constructed whose classes are linearly separable, but where gradient descent will not determine a hyperplane that classifies all examples correctly (whereas the *PT* Algorithm of course does).

©STEIN 2005-2020